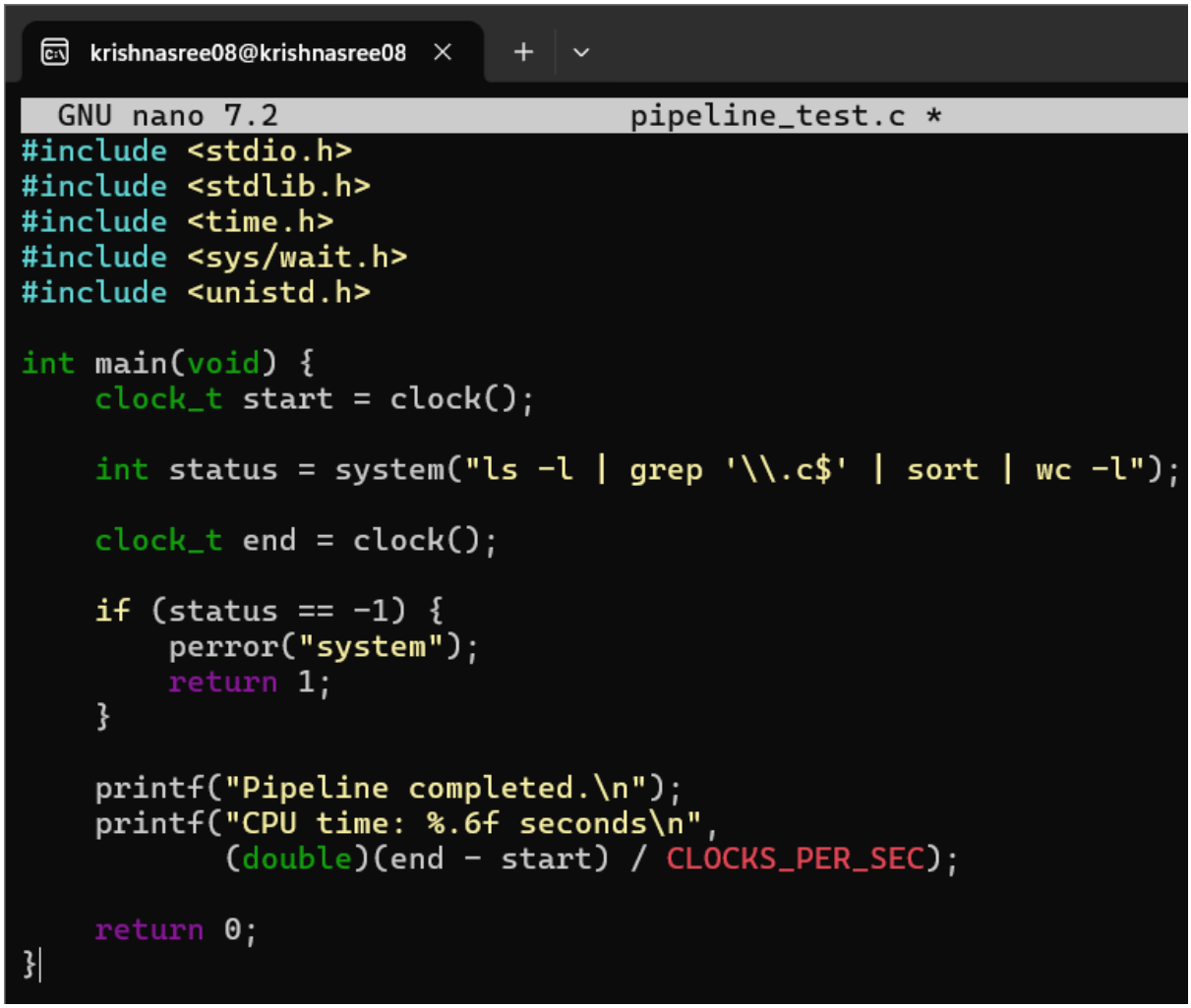
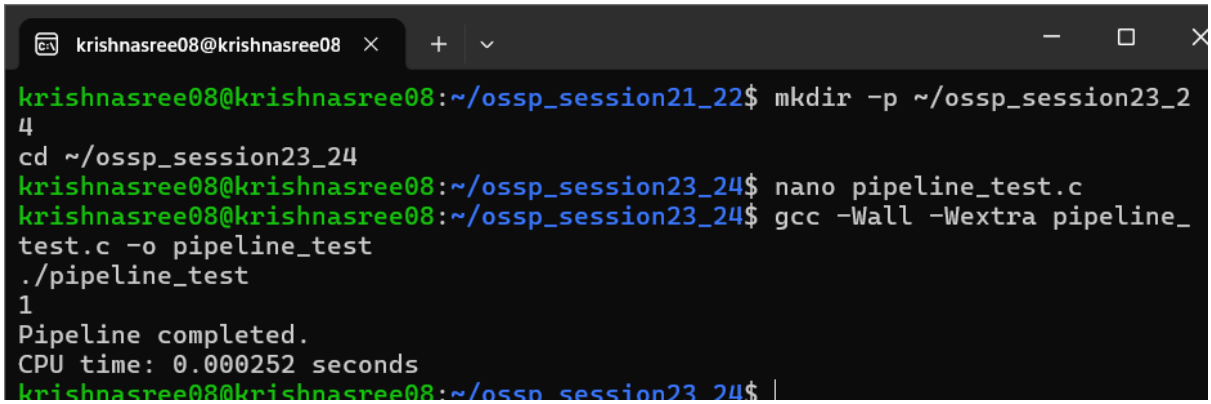
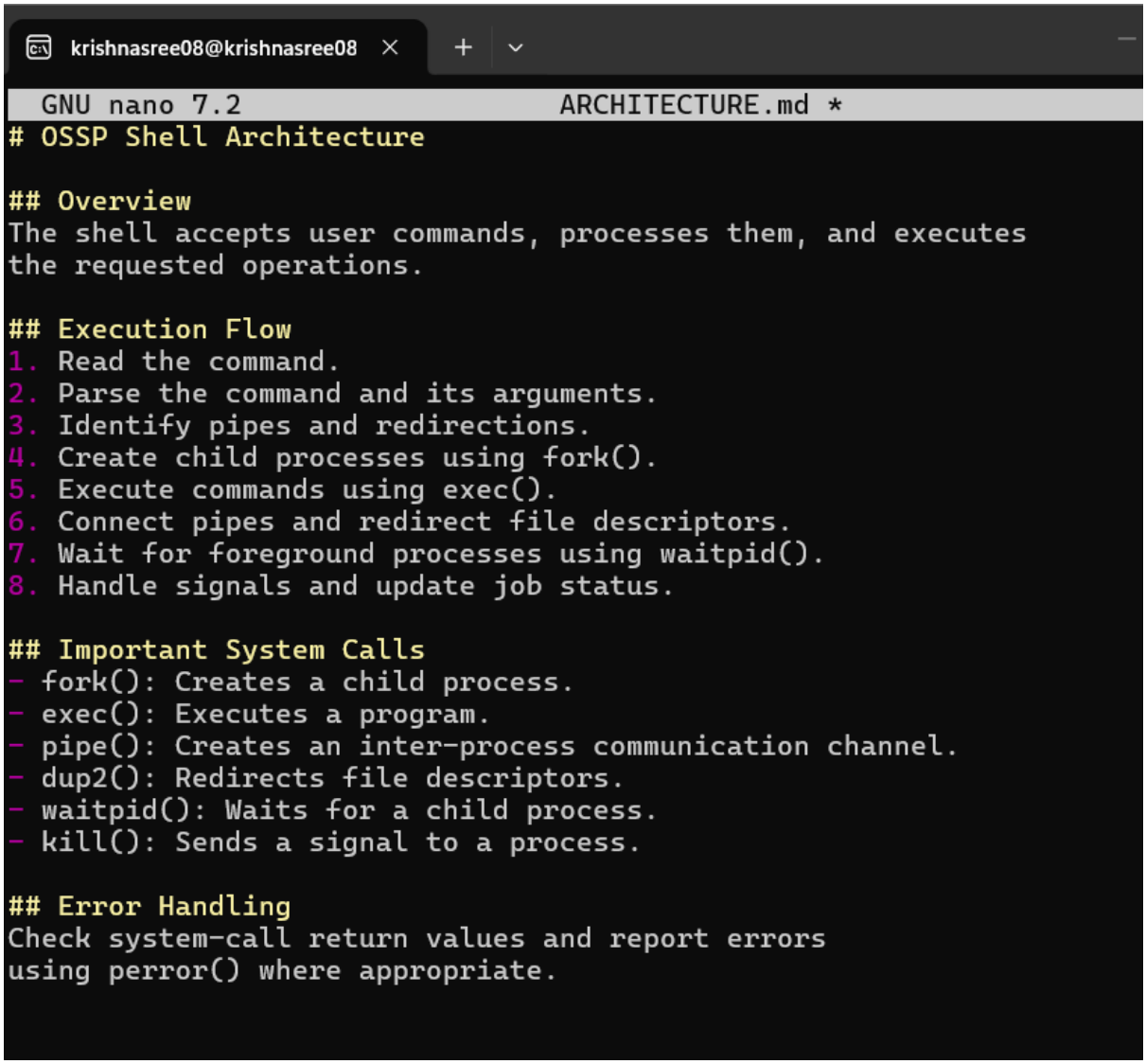

krishnasree08@krishnasree08
GNU nano 7.2 pipeline_test.c *
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <sys/wait.h>
#include <unistd.h>
int main(void) {
clock_t start = clock();
int status = system("ls -l | grep '\\.c$' | sort | wc -l");
clock_t end = clock();
if (status == -1) {
perror("system");
return 1;
}
printf("Pipeline completed.\n");
printf("CPU time: %.6f seconds\n",
(double)(end - start) / CLOCKS_PER_SEC);
return 0;
}


krishnasree08@krishnasree08
krishnasree08@krishnasree08:~/ossp_session21_22$ mkdir -p ~/ossp_session23_24
cd ~/ossp_session23_24
krishnasree08@krishnasree08:~/ossp_session23_24$ nano pipeline_test.c
krishnasree08@krishnasree08:~/ossp_session23_24$ gcc -Wall -Wextra pipeline_test.c -o pipeline_test
./pipeline_test
1
Pipeline completed.
CPU time: 0.000252 seconds


krishnasree08@krishnasree08
GNU nano 7.2 ARCHITECTURE.md *
OSSP Shell Architecture
Overview
The shell accepts user commands, processes them, and executes
the requested operations.
Execution Flow
1. Read the command.
2. Parse the command and its arguments.
3. Identify pipes and redirections.
4. Create child processes using fork().
5. Execute commands using exec().
6. Connect pipes and redirect file descriptors.
7. Wait for foreground processes using waitpid().
8. Handle signals and update job status.
Important System Calls
- fork(): Creates a child process.
- exec(): Executes a program.
- pipe(): Creates an inter-process communication channel.
- dup2(): Redirects file descriptors.
- waitpid(): Waits for a child process.
- kill(): Sends a signal to a process.
Error Handling
Check system-call return values and report errors
using perror() where appropriate.

```
krishnasree08@krishnasree08:~/ossp_session23_24$ nano ARCHITECTURE.md
krishnasree08@krishnasree08:~/ossp_session23_24$ cat ARCHITECTURE.md
# OSSP Shell Architecture

## Overview
The shell accepts user commands, processes them, and executes
the requested operations.

## Execution Flow
1. Read the command.
2. Parse the command and its arguments.
3. Identify pipes and redirections.
4. Create child processes using fork().
5. Execute commands using exec().
6. Connect pipes and redirect file descriptors.
7. Wait for foreground processes using waitpid().
8. Handle signals and update job status.

## Important System Calls
- fork(): Creates a child process.
- exec(): Executes a program.
- pipe(): Creates an inter-process communication channel.
- dup2(): Redirects file descriptors.
- waitpid(): Waits for a child process.
- kill(): Sends a signal to a process.

## Error Handling
Check system-call return values and report errors
using perror() where appropriate.
```

```
krishnasree08@krishnasree08:~/ossp_session23_24$ mkdir -p src docs
nano src/main.c
krishnasree08@krishnasree08:~/ossp_session23_24$ nano Makefile
krishnasree08@krishnasree08:~/ossp_session23_24$ make
./ossp_demo
make clean
gcc -Wall -Wextra src/main.c -o ossp_demo
OSSP shell project is running.
rm -f ossp_demo
krishnasree08@krishnasree08:~/ossp_session23_24$ nano demo.sh
krishnasree08@krishnasree08:~/ossp_session23_24$ chmod +x demo.sh
./demo.sh
=== OSSP Project Demo ===
1. Pipeline demonstration:
apple
banana
orange

2. Signal demonstration:
Signal-related practicals were completed separately.

3. Build verification:
gcc -Wall -Wextra src/main.c -o ossp_demo
Build successful.
OSSP shell project is running.

Demo completed.
```

Overall result — Sessions 23 & 24

Thus, the practicals demonstrated multi-command pipeline execution, basic performance measurement, architecture documentation, source-code organization, automated builds, and final project validation. These activities helped improve understanding of shell execution, process management, project maintainability, and software demonstration.